

Scoping, context

Brainster Web Development Academy



Table of contents	01	Terminology: Context and scope
	02	The global scope
	03	Local scopes
	04	(Fat) Arrow functions
	05	Date object
	06	Date methods
	07	Time methods



Terminology: Context and Scope

- To put it simply - context is the value of **this**. In other words, who “owns” the code that currently executes.
- Scope refers to which variables / properties / methods a function can access when it executes.
- In a way, you can think of scope as referring to functions and context as referring to objects. Not always strictly true, but it explains it for the most part.



The global scope

- The top-most scope is called the global scope.
- The global scope is available everywhere (but that's not always a good thing!)
- In most JavaScript environments, the global scope can be accessed by using the window object.
- Node.js is the exception here - the global scope is simply known as global.

3

```
const globalVar = 'global';  
  
function func() {  
    console.log(globalVar); // 'global', available  
}  
  
console.log(globalVar); // 'global', available
```



Local scopes

- We can divide local scopes in two categories - **function scope** and **block scope**.
- When you declare a variable in a function, you can access this variable only within the function (unless you **return** it).
- A “block” is anything with an opening and a closing curly brace - if, switch, for... As with function scope, you can access a variable only in its own scope (block).



Local scopes - functions

```
function func() {  
  const localVar = 'local';  
  console.log(localVar); // 'local', available  
}  
  
console.log(localVar); // undefined, unavailable
```



Local scopes - blocks

```
{ // This is valid JS ;)  
  const localVar = 'local';  
  console.log(localVar); // 'local', available  
}
```

```
console.log(localVar); // undefined, unavailable
```

```
// Same goes for if, switch, for and anything with curly braces
```



(Fat) Arrow functions

- Arrow functions are an exception to the context rules - they don't come with a this of their own.
- They inherit the this from the nearest context (class, object, etc.)
- Never rely on this in an arrow function :)



Takeaway

- Keep variables as local as possible.
- Avoid using global variables.
- Avoid using this unless you absolutely need to use it (and fully understand how it behaves under different conditions).
- Bonus points: this can also be rewritten! Read a bit on call, apply and bind if you want to find out more.



Aside: var vs let

Although let is the newer and more used way of declaring variables, var is still present. Here are some differences between them:

Scope:

- **var:** Function-scoped. This means if you declare a variable with var inside a function, it's only available within that function. However, if declared outside any function, it becomes a global variable.
- **let:** Block-scoped. Variables declared with let are only available within the nearest set of curly braces (e.g., inside loops, conditionals).

Hoisting:

- **var:** Variables declared with var are hoisted to the top of their function or script, but they are initialized with undefined.
- **let:** Variables declared with let are also hoisted to the top of their block, but they are not initialized, leading to a Temporal Dead Zone (TDZ). Accessing them before their declaration in the code will result in a ReferenceError.



Aside: var vs let

Re-declaration:

- **var:** Allows re-declaration of the same variable name in the same scope without any errors.
- **let:** Does not allow re-declaration of the same variable name in the same scope.

Global Object Property:

- **var:** When declared in the global scope, variables created with var become properties of the global object (e.g., window in browsers).
- **let:** When declared in the global scope, they do not become properties of the global object.

Initialization:

- **var:** Can be declared without initialization (defaults to undefined).
- **let:** Can also be declared without initialization.



Date object

Brainster Web Development Academy



Date object - terminology

- The Date object is a built-in object in JavaScript that stores the date and time. It provides a number of built-in methods for formatting and managing that data.
- By default, a new Date instance without arguments provided creates an object corresponding to the current date and time.

7

```
const currentDate = new Date();  
console.log(currentDate);
```

```
// Sat Aug 19 2023 19:59:31 GMT+0200 (Central European Summer Time)
```



Date methods

- `currentDate.getFullYear();`
// returns the current year - 2020
- `currentDate.getMonth();`
// returns the current month - 1 (february) in a range of 0-11 where january is 0.
- `currentDate.getDate();`
// returns the current day of the month - 25 in a range of 1-31 where 1 is the first of the month.
- `currentDate.getDay();`
// returns the current day of the week - 2 in a range of 0-6 where 0 is sunday.



Time methods

- `currentDate.getHours();`
// returns the current hour on the clock 16 (4 pm) in a range of 0-23 where 0 is midnight
- `currentDate.getMinutes();`
// returns the current minutes on the clock - 14
- `currentDate.getSeconds();`
// returns the current seconds on the clock - 55
- `currentDate.getMilliseconds();`
// returns the current milliseconds on the clock - 59



Exercise 1

1. Write a JavaScript function to get the month name from a given date.
2. Show the month name in console.

example:

given date: '2020 03 28'

console: March

*hint: use an array with all the month names.



Exercise 2

1. Write a JavaScript function to test whether a date is a weekend.
2. Show the result in console.

example:

given date: '2023 08 19'

console: it's the weekend.

7

given date: '2020 08 24'

console: it's not the weekend.



Exercise 3

1. Use the same functions from exercise I and exercise II to check from the current date what is the name of the current month and is it the weekend?
2. Show the result in console.



Homework

1. Refactor the functions from Exercise I and Exercise II to be methods to the Date object.

